

Use Case Diagram

Scenario

Scenario: Smart Parking System

Background

A city is introducing a **smart parking system** to manage parking spaces in busy urban areas. The goal is to reduce traffic congestion and make parking more efficient.

Drivers can use a mobile app or on-site machines to find and pay for parking. Parking spots are equipped with sensors that detect whether they are occupied or free.

Enforcement officers monitor parking usage and issue fines when violations occur.

Description

The system should support the following:

- People arriving in the city can search for available parking spots nearby.
- A user can reserve a parking spot for a limited time before arrival.
- When a vehicle occupies a parking spot, the system detects it automatically.
- Users must pay for parking based on the duration of their stay.
- Payments can be made through an external payment service.
- Users can extend their parking time remotely if needed.
- If a user overstays or parks without payment, the system flags a violation.
- Enforcement personnel can review violations and issue fines.
- The system keeps track of parking history and usage data.
- Administrators can manage parking zones, pricing rules, and system settings.

PlantUML Code

```
@startuml
actor Driver
actor Officer
actor Administrator
```

```
actor PaymentService
actor ParkingSystem

rectangle "Smart Parking System" {
    (Search)
    (Reserve)
    (Detect Vehicle)
    (Payment)
    (Extend)
    (Check Violations)
    (Issue Fine)
    (Maintain History)
    (System Management)

    Driver --> (Search)
    Driver --> (Reserve)
    Driver --> (Extend)
    Driver --> (Payment)

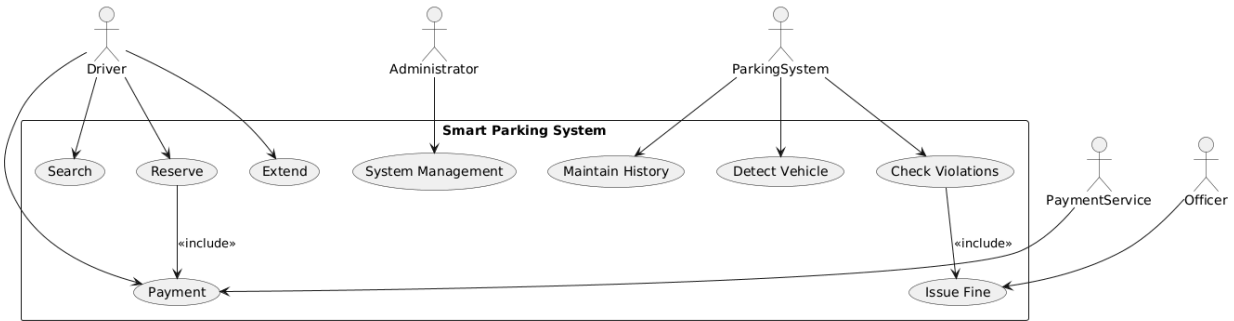
    Officer --> (Issue Fine)

    Administrator --> (System Management)

    PaymentService --> (Payment)

    ParkingSystem --> (Detect Vehicle)
    ParkingSystem --> (Check Violations)
    ParkingSystem --> (Maintain History)

    (Reserve) --> (Payment) : <<include>>
    (Check Violations) --> (Issue Fine) : <<include>>
}
@enduml
```



Sequence Diagram

Background

A company is building a food delivery platform similar to Wolt or Uber Eats.

Customers can order food from nearby restaurants, pay through the app, and track delivery in real time.

Scenario Description

A user opens the app and places an order from a restaurant.

The system should:

- Allow the user to select items and confirm the order
- Check that the restaurant is currently open
- Verify that all selected items are available
- Calculate the total price including delivery fees
- Process the payment using an external payment service
- Send the order to the restaurant for preparation
- Assign a delivery courier
- Notify the user about order status updates

Alternative Flows (Important!)

Your sequence diagram must handle at least one of these:

- The restaurant is closed
 - One or more items are unavailable
 - Payment fails
 - No courier is available
-

Behavioral Notes

- Some steps may happen **in parallel**
- Some steps may require **waiting for a response**
- Notifications may be sent **asynchronously**
- External systems are involved

PlantUML Code

```
@startuml
actor User
participant "Walt App" as Walt
participant "Restaurant" as Restaurant
database "Items Database" as DB
participant "Courier Service" as Courier
participant "Credit Card Company" as Payment

loop for each selected item
    User -> Walt: Select Item
    Walt -> Restaurant: isOpen
    Restaurant -> Walt: open
    Walt -> Restaurant: itemSelected
    Restaurant -> DB: checkItem
    DB -> Restaurant: itemExists
    Restaurant -> Walt: itemExists
end

Walt -> Courier: getDeliveryCost
Walt -> Walt: calculateTotal
Walt -> User: total
User -> Walt: payWithCreditCard
Walt -> Payment: pay

alt Payment approved
    Payment -> Walt: success
    Walt ->> Restaurant: prepareOrder
```

```
    Walt ->> Courier: assignCourier
    Restaurant -> Walt: orderStatus
    Courier -> Walt: deliveryStatus
    Walt ->> User: statusUpdate
else Payment failed
    Payment --> Walt: Failed
    Walt -> User: Show payment error
end
```

```
Payment -> Walt: success
Walt ->> Restaurant: prepareOrder
Walt ->> Courier: assignCourier
Restaurant -> Walt: orderStatus
Courier -> Walt: deliveryStatus
Walt ->> User: statusUpdate
```

@enduml

